

Implement SQLite GitSource storage

- Project: sportmate-interview
- Storage: active
- Session ID: 01a0fc0d-7ebd-7971-a5fe-97e6c4553efc
- Started: 2026-10-02T09:58:58.067Z
- Updated: 2026-10-02T14:21:13.753Z
- Model: gpt-6-astra

This PDF is a classified, derived reading view. Raw JSONL remains the canonical lossless representation when included.

User – 2026-10-02T09:59:01.293Z

A feladatod lecserélni a mock adatokat a frontendről, és tényleges backend SQLite mentést megtervezned, majd implementálnod. Jelenleg a frontenden a Sources-ek mind a [git-sources.ts \[blocked invalid-link\]](#) -ből jönnek, típusuk a [git-source.ts \[blocked invalid-link\]](#), ezek töltenek be a bal oldali navbar-ba.

A navbar célja, és api:

- A bal oldali navbarba az elmentett GitSource-ok jelenjenek meg.
- Pagination-t csinálni arra az eshetőségre, ha X darabnál több GitSource létezik.
- A "New Source" gomb felett legyen a szokásos pagination navigation rész: balra, jobbra, translation a jelenleg megjelenített mennyiségről és az össz mennyiségről.
- Sütiben tároljuk azt, hogy melyik oldalon jártunk.
- Mindig csak az aktuális oldalnyi gitsource-t query-zzuk az sqlite adatbázisból. Lapozás esetén amíg betöltenek az adatok megjelenik egy loader, animált ikon amíg az új lekérés teljesül.

A New Source gomb és modal működése:

- Kliens oldalon már most vannak létező, alap validálások (például üres karakter stb), ezt természetesen szerver oldalon is meg kell tenned. Fontos, hogy ne duplikáld a validálásokat, tehát ne írd meg 2x ugyanazt a checkolást a szerver oldalra és a kliens oldalra egyaránt, hanem ezeket központosítsd: ha egy helyen átirom a validálást, az szerver oldalon és kliens oldalon is megváltozik. Ez amolyan "zed like" integráció. Ha erre a jelenlegi stack nem ad könnyen lehetőséget, akkor jelezd nekem és átbeszéljük.
- A kiválasztott provider-nek a backendjében (commit: 531bd0825457690708938a2a9721c7ffaa062a11) kell egy olyan metódus, ami azt adja vissza, hogy a megadott string az egy létező, valid user vagy organization. Ezt a validálást a létrehozás endpointjába integrálnod kell, hogy csak olyan felhasználót lehessen hozzáadni, ami valós. Ezen felül fontos, hogy minden GitProvider-nek lennie kell egy olyan methodjának ami kap egy string-et, és vissza adja azt, hogy az az adott platformra egy valid

regex alapú lehetőség. Mire gondolok ez alatt?: Keresd ki, hogy a Github regisztrációkor vagy Organization létrehozásakor milyen regex checket használ pontosan, és a github implementációban pontosan erre a regexre kell validálnia még az api kérés küldése előtt. Erre azért van szükség, mert feltételeznünk kell, hogy ahány GitProvider, annyi különböző szabályozás.

Egy GitSource adatmodellje, ehhez bővítsd ki a jelenlegit.

- Név
- URL
- (Profilkép) -> ehhez bővítened kell a gitsource és a gitprovider backendet, hogy egy [GitProvider.php \[blocked invalid-link\]](#)-től megkaphassuk (ha támogat ilyet az adott provider/provider apija (github biztosan támogat))
- AccountType: user | organization
- Last sync timestamp

Fontos, hogy el kell készítened a backend logikát a lekérések és a CREATE teljesítéséhez. Ezt a backend service-t egy REST API fogja használni, a REST api-t pedig a kliens.

Bármilyen hiba esetén egy toast-al jelezzük a felhasználónak, hogy sikertelen volt az, amit szeretett volna csinálni (pl.: már létezik az adott source, hiba volt a lekérdezésben stb). Ezek a hiba üzenetek mind legyenek az [en.ts \[blocked invalid-link\]](#) fájlban.

Mielőtt elkezdenéd a tervezést vagy az implementálást, olvass be mindent, gyűjts elegendő kontextust, és tegyél fel nekem benned felmerülő kérdéseket ha van. Ha ezeket megválaszoltam, akkor készítsd el a tervet.

Assistant question – 2026-10-02T10:00:51.139Z

Ris, elfogadható a Precognition hálózati validálása, vagy szükséges a böngészőben önállóan futó ellenőrzés? Például az „acme--team” hibáját az első megoldásnál a Laravel válasza jelzi, a másodiknál helyben derül ki.

- Precognition (Recommended): A szabályokat egyszer, PHP-ban definiáljuk; például az „acme--team” hibáját gépelési szünet után háttérkérés jelzi, GitHub-hívás nélkül.
- Közös helyi séma: Közös sémából futtatunk PHP- és böngészős ellenőrzést; például az „acme--team” hálózat nélkül is hibás, de ehhez külön sémaintegráció szükséges.

Ris, mit jelentsen a GitSource „Név” mezője? Például a „vuejs” azonosítóhoz a provider külön megjelenített nevet is adhat, és saját címke is elképzelhető.

- Provider neve (Recommended): A provider megjelenített nevét mentjük, hiányában a „vuejs” azonosítót; a külön account mezőben mindig megmarad a „vuejs”.
- Fiókazonosító: A név mindig a kanonikus account, például „vuejs”; külön megjelenített nevet vagy saját címkét nem kérünk.

- Saját elnevezés: A modal külön névmezőt kap; például a „vuejs” forrás saját neve „Frontend referenciák” lehet.

Ris, mit jelentsen most a „Last sync timestamp”? Például a „vuejs” létrehozásakor a profilját ellenőrizzük, de a repository-lista szinkronizálása külön művelet lenne.

- Későbbi reposzinkron (Recommended): Létrehozásakor null értéket mentünk és „Never synced” állapotot mutatunk; a „vuejs” repository-szinkronja későbbi feladat marad.

- Profil lekérésének ideje: A „vuejs” sikeres profil-lekérésének idejét mentjük; repository-kat ebben a feladatban nem szinkronizálunk.

- Reposzinkron is szükséges: A „vuejs” hozzáadása repository-k lekérését és mentését is elindítja; a tervet erre a további backend működésre is kiterjesztjük.

User answer – 2026-10-02T10:01:25.065Z

source_name: Provider neve (Recommended)

validation_strategy: Precognition (Recommended)

sync_scope: Későbbi reposzinkron (Recommended)

Assistant question – 2026-10-02T10:01:47.443Z

Ris, hány GitSource jelenjen meg egy oldalon? Például 23 mentett forrásnál 10-es oldalmérettel három oldal lenne, és a második oldal „11–20 / 23” számlálót mutatna.

- 10 forrás (Recommended): 23 forrásból három oldal lesz; a backend egyszerre legfeljebb 10 GitSource rekordot olvas be.

- 5 forrás: 23 forrásból öt oldal lesz; a navbar listája rövidebb marad, de gyakrabban kell lapozni.

- 20 forrás: 23 forrásból két oldal lesz; több forrás látszik egyszerre, kisebb képernyőn hosszabb görgetéssel.

Ris, a navbar meglévő keresője az összes elmentett forrásban keressen, vagy csak az aktuális oldalon? Például az első oldalon beírt „laravel” találat a harmadik oldalon is szerepelhet.

- Összes forrás (Recommended): A backend a teljes SQLite-listában keres, és csak a találatok aktuális oldalát küldi vissza; a harmadik oldali „laravel” is megtalálható.

- Aktuális oldal: A keresés csak a már betöltött oldalt szűri; a harmadik oldali „laravel” az első oldalról nem jelenik meg.

Ris, milyen sorrendben jelenjenek meg a források, és hová kerüljön az új elem? Például a második oldalon hozzáadott „vuejs” után az új forrást azonnal meg tudjuk mutatni.

- Legújabb elöl (Recommended): Létrehozási idő szerint csökkenően rendezünk; új „vuejs” mentése után az első oldalra lépünk, töröljük a keresést, és kijelöljük az új elemet.

- Név szerinti sorrend: Név szerint rendezünk; új „vuejs” mentése után töröljük a keresést, és arra az oldalra lépünk, ahol az új elem betűrendben található.

User answer – 2026-10-02T10:03:04.976Z

search_scope: A keresőt még nem kell implementálnod.

list_order: Legújabb elől (Recommended)

page_size: 10 forrás (Recommended)

Assistant question – 2026-10-02T10:03:26.230Z

Ris, a mai regisztrációs szabályokat kövessük szigorúan, akkor is, ha emiatt egy létező régi account kimarad? Például az „Artur-” létezik, de a záró kötőjel ma nem megengedett.

- Szigorú szabály (Recommended): Az „Artur-” nevet GitHub-kérés előtt elutasítjuk; az új regisztráció dokumentált szintaxisát alkalmazzuk.

- Régi fiókok támogatása: Az „Artur-” és az enterprise-fiókok ismert kivételeit is megengedjük az előellenőrzésben; a tényleges létezést a GitHub API dönti el.

Ris, mi történjen a már meglévő keresőmezővel, amíg a keresés nincs implementálva? Például a harmadik oldalon tárolt „laravel” forrást a jelenlegi helyi szűrés az első oldalról nem találná meg.

- Ideiglenesen elrejtjük (Recommended): A keresőmező most nem jelenik meg; a „laravel” forrást lapozással lehet elérni, amíg később elkészül a teljes keresés.

- Helyi szűrés marad: Megőrizzük a jelenlegi szűrést az aktuális oldal 10 elemére; az első oldalról a harmadik oldali „laravel” nem kereshető.

Ris, az új mezők hol jelenjenek meg? Például a „vuejs” forrásnál elérhető lesz profilkép, profil-URL, organization típus és „Never synced” állapot.

- Részletpanel és avatar (Recommended): A navbarban név és avatar látszik; a kijelölt forrás paneljén a „vuejs” azonosító, kattintható profil-URL, típus és szinkronállapot is megjelenik.

- Csak tárolás és API: A mezőket mentjük és visszaadjuk az API-ban; a „vuejs” navbar- és részletnézete a mostani minimális megjelenést őrzi meg.

User answer – 2026-10-02T10:04:31.325Z

metadata_display: Részletpanel és avatar (Recommended)

github_name_policy: Szigorú szabály (Recommended)

existing_search_ui: Helyi szűrés marad

Final report – 2026-10-02T10:06:29.122Z

<proposed_plan>

Ris, **GitSource SQLite-mentés és REST-integráció**

Rögzített működés

A mock listát SQLite-ban tárolt források váltják. Oldalanként ****10 elem****, legújabb elől. Új forrás létrehozása után első oldal, üres kereső és az új elem kijelölése.

A név a provider megjelenített neve, hiányában a fiókazonosító. `last\_synced\_at` kezdetben `null`; tényleges repository-szinkron későbbi feladat. A meglévő kereső marad, kizárólag az aktuális oldal elemeit szűri.

Backend és provider

- Új `App\Models\GitSource` Eloquent modell és migráció. Mezők: provider azonosító, upstream account ID, kanonikus account, normalizált account, név, profil-URL, nullable avatar-URL, `AccountType`, nullable `last\_synced\_at`, timestamps.

- Adatbázisszintű egyediség provider + upstream ID, illetve provider + normalizált account szerint. A GitHub-nevek kis- és nagybetűs változatai ugyanannak számítanak. Párhuzamos létrehozáskor is legfeljebb egy rekord keletkezhet.

- A meglévő `App\Git\GitSource` domain objektum külön marad. Új metadata gettereket kap; jelenlegi `getName()` továbbra is a kanonikus accountot jelenti, így a repository-lekérések kompatibilisek maradnak.

- A `GitProvider` új `isValidAccountName(string): bool` metódust kap. A FormRequest és a provider ugyanazt az ellenőrzést használja.

- GitHub-nál szigorú szabály: 1–39 ASCII alfanumerikus karakter vagy egyszeres kötőjel; kezdő/záró kötőjel tiltott. Ez a dokumentált regisztrációs szabály megfelelője, nem igazoltan a GitHub belső regexének másolata. Régi és enterprise-kivételek kizárva, a választásod szerint. [GitHub signup \(https://github.com/signup\)](https://github.com/signup)

- A kibővített `getSource()` egyetlen `/users/{account}` kéréssel ellenőrzi a létezést, és visszaadja a kanonikus accountot, upstream ID-t, nevet, URL-t, avatart és típust. Csak `User` és `Organization` fogadható el. [GitHub REST dokumentáció \(https://docs.github.com/en/rest/users/users#get-a-user\)](https://docs.github.com/en/rest/users/users#get-a-user)

- Külön `GitSourceService` végzi a listázást és létrehozást. A controller HTTP-koordinációt végez; külső API-kérés nem fut nyitott adatbázis-tranzakcióban.

REST API és közös validálás

- `GET /api/git-sources?page=N`: JSON `data` és lapozási metadata: `current\_page`, `last\_page`, `per\_page`, `total`. Rendezés: `created\_at DESC, id DESC`. Összesítő COUNT mellett kizárólag az aktuális oldal rekordjai kerülnek betöltésre.

- `POST /api/git-sources`: bemenet `{ provider, account }`; siker esetén `201` és a mentett forrás. A metadata kizárólag a provider válaszából származik.

- A JSON-forrás típusa szerializálható adatokat tartalmaz: `id`, `provider`, `account`, `name`, `url`, `avatar\_url`, `account\_type`, `last\_synced\_at`. A provider ikonja külön frontend registryben marad.

- Same-origin JSON endpointok a Laravel `web` middleware-rel, meglévő session- és CSRF-védelemmel. Új autentikáció nem készül.
- Egyetlen FormRequest tartalmazza a kötelező mezőket, provider-ellenőrzést, szintaxist és duplikációellenőrzést. A frontend kézzel írt regex-, hossz- és duplikációellenőrzése megszűnik.
- Inertia `useHttp` + Precognition ad élő mezővalidálást, 400 ms késleltetéssel. Ez nem ment adatot és nem hív GitHubot; a távoli létezésellenőrzés a tényleges CREATE során fut.
[Precognition dokumentáció \(https://laravel.com/framework/docs/13.x/precognition\)](https://laravel.com/framework/docs/13.x/precognition)
- Validációs, nem létező account és duplikációs hibák: `422`, mezőhibákkal. Provider rate limit: `429`; hibás provider-válasz: `502`; elérhetetlenség: `503`. Váratlan mentési/listázási hiba naplózva, általános felhasználói hibakóddal.
- Minden megjelenített hiba és fallback az `en.ts` fájlban lesz. A backend gépi hibakódokat/fordítási kulcsokat küld; nyers exception-szöveg nem jelenik meg.
- A projekt skillje szerint Wayfinder kerül integrálásra: Composer-csomag, Vite-plugin és generált route-hívások. [Wayfinder dokumentáció \(https://github.com/laravel/wayfinder\)](https://github.com/laravel/wayfinder)

Frontend, pagination és hibakezelés

- Első betöltés és lapozás REST-kéréssel történik. A frontend kizárólag az aktuális oldalt és külön a kijelölt forrás objektumát tartja meg.
- A New Source felett bal/jobbs lapozógomb és fordítható számláló jelenik meg. A számláló a ténylegesen látható elemek számát és az összes mentett forrás számát mutatja; helyi szűrőskor is.
- Betöltés alatt animált ikon, `aria-busy` és tiltott lapozógombok. Sikertelen lapváltáskor az előző oldal megmarad; első betöltési hibánál újrapróbálkozás elérhető. Elavult válasz nem írhatja felül az új állapotot.
- Az oldalt szerver által kezelt, egyéves `git\_sources\_page` cookie tárolja: HttpOnly, SameSite=Lax, HTTPS-en Secure. Csak sikeres listaválasz módosítja. Hibás érték első oldalra, túl nagy érték az utolsó létező oldalra áll vissza.
- Lapozáskor a kiválasztás megmarad. Teljes újratöltéskor a visszaállított oldal első eleme lesz kijelölve; üres listánál nincs kiválasztás.
- Mentés közben dupla beküldés és modalbezárás tiltva. Hibánál a modal és bevitt érték megmarad; siker után bezár, az új forrás kijelölődik, az első oldal újratöltődik. Ha csak ez az újratöltés hibázik, külön listafrissítési hibát jelzünk; a sikeres mentést megőrizzük.
- Navbarban név és avatar; hiányzó vagy hibás képnél provider ikon. Részletpanelen account, kattintható profil-URL, fióktípus és „Never synced” állapot.
- Közös toast-kezelés készül. Sikertelen művelet toastot kap, validációs hibánál mezőhiba is megjelenik. Gépelés közbeni validáció csak mezőhibát mutat; hálózati hibája toastot is. A toast a legfelső aktív dialogban jelenik meg, mobilon is.
- A mock adatok és használatuk megszűnnek. A meglévő mobil drawer, fókuszkezelés és témák megmaradnak.

Ellenőrzés és alapértelmezések

- **Pest + SQLite:** user/organization létrehozás és mentett metadata; névfallback, avatar hiánya; kisbetűs/nagybetűs és upstream ID szerinti duplikáció; adatbázis-egyediségi ütközés kezelése.
- **Validáció/provider:** hibás bemenetnél nulla külső kérés; Precognitionnál nulla mentés és GitHub-hívás; normál létrehozásnál egy account-lookup; not-found, timeout, rate limit és hibás válasz esetén nincs rekord.
- **Pagination:** 0/10/11/23 elem, stabil rendezés, oldalméret, totals, cookie-visszaállítás és hibás cookie. Ellenőrzés arra is, hogy a lista lekérése SQL-szinten korlátozott.
- **Böngészős ellenőrzés:** desktop/mobil, loader, hibatoast modal felett, újrapróbálkozás, kijelölés megőrzése, helyi szűrés, dupla submit, billentyűzet és fókusz.
- **Projektellenőrzések:** célzott tesztek, majd teljes Pest-suite, Pint, PHPStan, frontend lint/formázásellenőrzés, TypeScript és production build. Új tesztfüggőség nem szükséges.
- **Alapértelmezés:** globális közös forráslista a jelenlegi alkalmazás szerint; csak GitHub provider. Repository-szinkron, szerveroldali keresés, szerkesztés és törlés kimarad. Az adatbázis üresen indul, mock seed nélkül; meglévő adatokat megőrző migráció készül.

</proposed_plan>

User – 2026-10-02T10:07:18.024Z

PLEASE IMPLEMENT THIS PLAN:

Ris, **GitSource SQLite-mentés és REST-integráció**

Rögzített működés

A mock listát SQLite-ban tárolt források váltják. Oldalanként **10 elem**, legújabb elől. Új forrás létrehozása után első oldal, üres kereső és az új elem kijelölése.

A név a provider megjelenített neve, hiányában a fiókazonosító. `last\_synced\_at` kezdetben `null`; tényleges repository-szinkron későbbi feladat. A meglévő kereső marad, kizárólag az aktuális oldal elemeit szűri.

Backend és provider

- Új `App\Models\GitSource` Eloquent modell és migráció. Mezők: provider azonosító, upstream account ID, kanonikus account, normalizált account, név, profil-URL, nullable avatar-URL, `AccountType`, nullable `last\_synced\_at`, timestamps.
- Adatbázisszintű egyediség provider + upstream ID, illetve provider + normalizált account szerint. A GitHub-nevek kis- és nagybetűs változatai ugyanannak számítanak. Párhuzamos létrehozáskor is legfeljebb egy rekord keletkezhet.
- A meglévő `App\Git\GitSource` domain objektum külön marad. Új metadata gettereket kap; jelenlegi `getName()` továbbra is a kanonikus accountot jelenti, így a repository-lekérések

kompatibilisek maradnak.

- A `GitProvider` új `isValidAccountName(string): bool` metódust kap. A FormRequest és a provider ugyanazt az ellenőrzést használja.

- GitHubnál szigorú szabály: 1–39 ASCII alfanumerikus karakter vagy egyszeres kötőjel; kezdő/záró kötőjel tiltott. Ez a dokumentált regisztrációs szabály megfelelője, nem igazoltan a GitHub belső regexének másolata. Régi és enterprise-kivételek kizárva, a választásod szerint. [GitHub signup \(https://github.com/signup\)](https://github.com/signup)

- A kibővített `getSource()` egyetlen `/users/{account}` kéréssel ellenőrzi a létezést, és visszaadja a kanonikus accountot, upstream ID-t, nevet, URL-t, avatar és típust. Csak `User` és `Organization` fogadható el. [GitHub REST dokumentáció \(https://docs.github.com/en/rest/users/users#get-a-user\)](https://docs.github.com/en/rest/users/users#get-a-user)

- Külön `GitSourceService` végzi a listázást és létrehozást. A controller HTTP-koordinációt végez; külső API-kérés nem fut nyitott adatbázis-tranzakcióban.

REST API és közös validálás

- `GET /api/git-sources?page=N`: JSON `data` és lapozási metadata: `current\_page`, `last\_page`, `per\_page`, `total`. Rendezés: `created\_at DESC, id DESC`. Összesítő COUNT mellett kizárólag az aktuális oldal rekordjai kerülnek betöltésre.

- `POST /api/git-sources`: bemenet `{ provider, account }`; siker esetén `201` és a mentett forrás. A metadata kizárólag a provider válaszából származik.

- A JSON-forrás típusa szerializálható adatokat tartalmaz: `id`, `provider`, `account`, `name`, `url`, `avatar\_url`, `account\_type`, `last\_synced\_at`. A provider ikonja külön frontend registryben marad.

- Same-origin JSON endpointok a Laravel `web` middleware-rel, meglévő session- és CSRF-védelemmel. Új autentikáció nem készül.

- Egyetlen FormRequest tartalmazza a kötelező mezőket, provider-ellenőrzést, szintaxist és duplikációellenőrzést. A frontend kézzel írt regex-, hossz- és duplikációellenőrzése megszűnik.

- Inertia `useHttp` + Precognition ad élő mezővalidálást, 400 ms késleltetéssel. Ez nem ment adatot és nem hív GitHubot; a távoli létezésellenőrzés a tényleges CREATE során fut.

[Precognition dokumentáció \(https://laravel.com/framework/docs/13.x/precognition\)](https://laravel.com/framework/docs/13.x/precognition)

- Validációs, nem létező account és duplikációs hibák: `422`, mezőhibákkal. Provider rate limit: `429`; hibás provider-válasz: `502`; elérhetetlenség: `503`. Váratlan mentési/listázási hiba naplózva, általános felhasználói hibakóddal.

- Minden megjelenített hiba és fallback az `en.ts` fájlban lesz. A backend gépi hibakódokat/fordítási kulcsokat küld; nyers exception-szöveg nem jelenik meg.

- A projekt skillje szerint Wayfinder kerül integrálásra: Composer-csomag, Vite-plugin és generált route-hívások. [Wayfinder dokumentáció \(https://github.com/laravel/wayfinder\)](https://github.com/laravel/wayfinder)

Frontend, pagination és hibakezelés

- Első betöltés és lapozás REST-kéréssel történik. A frontend kizárólag az aktuális oldalt és külön

a kijelölt forrás objektumát tartja meg.

- A New Source felett bal/jobbs lapozógomb és fordítható számláló jelenik meg. A számláló a ténylegesen látható elemek számát és az összes mentett forrás számát mutatja; helyi szűrőkor is.

- Betöltés alatt animált ikon, `aria-busy` és tiltott lapozógombok. Sikertelen lapváltáskor az előző oldal megmarad; első betöltési hibánál újrapróbálkozás elérhető. Elavult válasz nem írhatja felül az új állapotot.

- Az oldalt szerver által kezelt, egyéves `git\_sources\_page` cookie tárolja: HttpOnly, SameSite=Lax, HTTPS-en Secure. Csak sikeres listaválasz módosítja. Hibás érték első oldalra, túl nagy érték az utolsó létező oldalra áll vissza.

- Lapozáskor a kiválasztás megmarad. Teljes újratöltéskor a visszaállított oldal első eleme lesz kijelölve; üres listánál nincs kiválasztás.

- Mentés közben dupla beküldés és modalbezárás tiltva. Hibánál a modal és bevitt érték megmarad; siker után bezár, az új forrás kijelölődik, az első oldal újratöltődik. Ha csak ez az újratöltés hibázik, külön listafrissítési hibát jelzünk; a sikeres mentést megőrizzük.

- Navbarban név és avatar; hiányzó vagy hibás képnél provider ikon. Részletpanelen account, kattintható profil-URL, fióktípus és „Never synced” állapot.

- Közös toast-kezelés készül. Sikertelen művelet toastot kap, validációs hibánál mezőhiba is megjelenik. Gépelés közbeni validáció csak mezőhibát mutat; hálózati hibája toastot is. A toast a legfelső aktív dialogban jelenik meg, mobilon is.

- A mock adatok és használatuk megszűnnek. A meglévő mobil drawer, fókuszkezelés és témák megmaradnak.

Ellenőrzés és alapértelmezések

- ****Pest + SQLite:**** user/organization létrehozás és mentett metadata; névfallback, avatar hiánya; kisbetűs/nagybetűs és upstream ID szerinti duplikáció; adatbázis-egyediségi ütközés kezelése.

- ****Validáció/provider:**** hibás bemenetnél nulla külső kérés; Precognitionnál nulla mentés és GitHub-hívás; normál létrehozásnál egy account-lookup; not-found, timeout, rate limit és hibás válasz esetén nincs rekord.

- ****Pagination:**** 0/10/11/23 elem, stabil rendezés, oldalméret, totals, cookie-visszaállítás és hibás cookie. Ellenőrzés arra is, hogy a lista lekérése SQL-szinten korlátozott.

- ****Böngészős ellenőrzés:**** desktop/mobil, loader, hibatoast modal felett, újrapróbálkozás, kijelölés megőrzése, helyi szűrés, dupla submit, billentyűzet és fókusz.

- ****Projektellenőrzések:**** célzott tesztek, majd teljes Pest-suite, Pint, PHPStan, frontend lint/formázásellenőrzés, TypeScript és production build. Új tesztfüggőség nem szükséges.

- ****Alapértelmezés:**** globális közös forráslista a jelenlegi alkalmazás szerint; csak GitHub provider. Repository-szinkron, szerveroldali keresés, szerkesztés és törlés kimarad. Az adatbázis üresen indul, mock seed nélkül; meglévő adatokat megőrző migráció készül.

Final report – 2026-10-02T10:19:37.285Z

Ris, kész az implementáció.

- SQLite-mentés, REST API, közös Precognition-validálás és GitHub-profiladatok.
- Lapozás, oldalsüti, loader, avatarok és hibatoastok.
- Migráció lefutott; saját adatbázisodban nincs tesztadat.
- ****174 teszt sikeres.**** Pint, PHPStan, frontend check, TypeScript és build zöld. Desktop- és mobilpróba kész.

A keresés továbbra is oldalankénti; repository-szinkron későbbi feladat.

Tesztadatbázison ellenőrzött felület:

!GitSource felület [local file not included]